

Inteligență Artificială

Bogdan Alexe

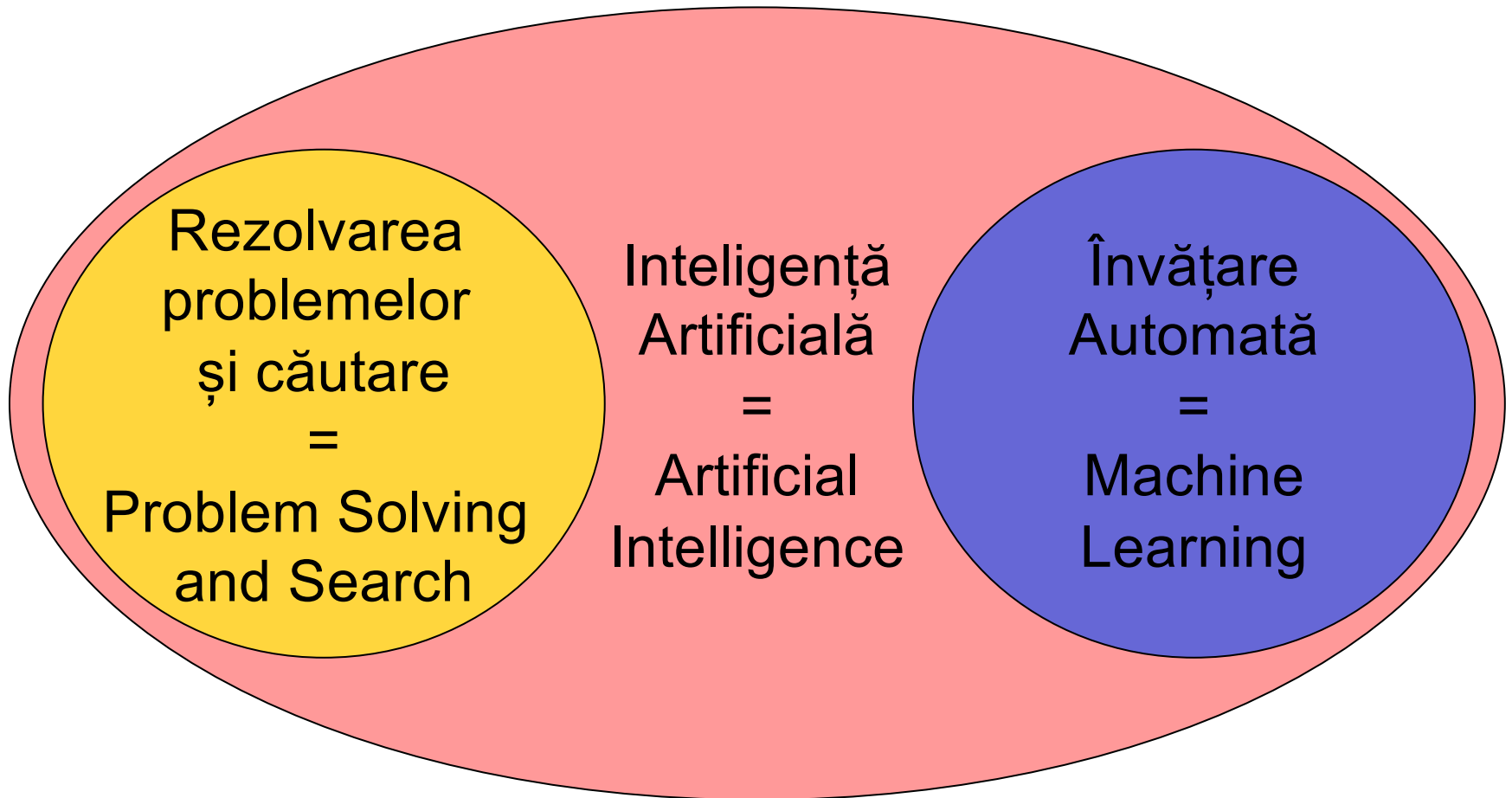
bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

Structura cursului de Inteligență artificială



Echipa noastră la laborator și seminar

Rezolvarea problemelor și căutare



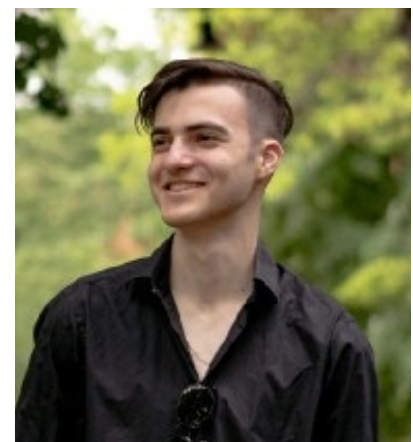
Irina

(săptămânile 8-14)
Grupele 241, 242



Miruna

(săptămânile 8-14)
Grupele 244



Ștefan

(săptămânile 8-14)
Grupa 243

Sistem de notare

Nota finală la disciplina Inteligență Artificială este formată din:

- nota de la curs $\mathbf{C} = (\mathbf{C}_1 + \mathbf{C}_2)/2$ (evaluare scrisă) 50%
 - la examenul scris din sesiune vor fi subiecte din ambele părți
 - media notelor ($\mathbf{C}_1, \mathbf{C}_2$) din fiecare parte (între 1 – 10)
- nota de la laborator $\mathbf{L} = (\mathbf{L}_1 + \mathbf{L}_2)/2$ (programare) 50%
 - vor fi două note la laborator, câte una pentru fiecare parte.
 - media notelor ($\mathbf{L}_1, \mathbf{L}_2$) din fiecare parte (între 1 – 10)
- nota finală se calculează ca media celor două note: nota de la curs și nota de la laborator = $(\mathbf{C} + \mathbf{L}) / 2$

Condiții de promovare: pentru a promova examenul trebuie ca $\mathbf{C} \geq 5$ și $\mathbf{L} \geq 5$ (regula se aplică și la restanță, **nu se reportează notele de la o sesiune la alta**)

Sistem de notare – Rezolvarea problemelor și căutare

- **bonus la Curs (C_1):**
 - se acordă conform clasamentului din Kahoot (organizat la sfârșit de curs)
 - seria 24 (120 de studenți) – locurile 1-4 obțin 0.3 puncte, locurile 5-8 obțin 0.2 puncte, locurile 9-12 obțin 0.1 puncte
 - până la 1.5 puncte în plus la nota din examen la partea de Rezolvarea problemelor și căutare (1.5 puncte din 10, fără a depăși nota 10)

Sistem de notare – Rezolvarea problemelor și căutare

- **Laborator (L_1):**

- studenții pot opta pentru una dintre următoarele variante de evaluare:
 - A = activitate pe parcurs: $1,5p / \text{laborator} \times 6 + 1p \text{ oficiu}$.
 - P = proiect: susținut în 3 etape ($3p / \text{etapă}$) + $1p \text{ oficiu}$.
- pentru studenții care nu au obținut punctajul prin activitate/proiect se va organiza un T = test de laborator înainte de ziua examinării scrise (săptămâna 14 sau sesiune)
- bonus din Seminar (S): se acordă maximum $1p$ bonus la nota de laborator pentru activitatea din cadrul seminarelor (la prima examinare)
- $L_1 = \min(10, \max(A, P, T) + S)$

Materiale

- pe MS-TEAMS, pe canalele Curs și Laborator + Seminar dedicate celor două părți

Inteligență Artificială - 2025-2026 FMI-UB

feua49v

Where do I enter the code?

Click **Join or create a team** below your teams list and look for the **Join a team with a code** card

< All teams

IA

Inteligență Artificială - 2025-2026 F...

...

Home page

Class Notebook

Classwork

Assignments

Grades

Insights

Reflect

▼ Main Channels

General

Învățare automată - curs

Învățare automată - lab și sem

Rezolvarea problemelor și căutare - curs

Rezolvarea problemelor și căutare - lab ...

IA

Rezolvarea problemelor și căutare - curs

Posts

Shared ▾

Notes

⊕

☰ All documents

💬 In messages

+ New ▾

⤴ Upload ▾

📄 Edit in grid view

🔗 Share

🔗 Copy link

📄 Forms

New

🔗 Add s...

Rezolvarea problemelor și căutare - curs

	Name ▾	Modified ▾	Modified By ▾	+ Add c...
	seria23	February 28	DUMITRU BOGDAN A	
	seria24	April 13	DUMITRU BOGDAN A	
	seria25	February 28	DUMITRU BOGDAN A	
	Artificial Intelligence - A modern approach ...	February 28	DUMITRU BOGDAN A	

Două paradigme fundamentale în Inteligența Artificială

Inteligența Artificială a evoluat istoric în jurul a două idei majore:

1. Paradigma **simbolică** în care inteligența = raționament explicit (căutare, planificare, jocuri)
 - ideea centrală: dacă știm regulile problemei, putem căuta soluția într-un spațiu al stărilor
→ Rezolvarea problemelor și căutare
2. Paradigma **statistică**, bazată pe date în care inteligența = învățare din experiență
 - ideea centrală: dacă nu știm regulile problemei, putem căuta soluția învățând din date →
Învățare automată

Ambele rezolvă același tip de problemă generală: cum construim agenți inteligenți.

Două paradigme fundamentale în Inteligența Artificială

La acest curs vom studia două moduri fundamentale prin care un sistem poate deveni inteligent:

- prin căutare într-un spațiu (de stări) bine definit.
- prin învățare din date atunci când regulile nu sunt cunoscute explicit.

În acest curs vom înțelege ambele mecanisme și vom vedea că, în esență, ele rezolvă aceeași problemă: optimizarea comportamentului unui agent într-un mediu.

Cuprinsul cursului de azi

1. Agenți inteligenți și mediile în care aceștia funcționează
2. Rezolvarea problemelor prin căutare
 - Formularea unei probleme
 - Graful stărilor
 - Arborele de căutare
 - Căutare într-un arbore de căutare
3. Căutare neinformată
 - Căutare în lățime (Bread-First Search)
 - Căutare în adâncime (Depth-First Search)
 - Căutare în adâncime limitată (Depth-Limited Search)
 - Căutare în adâncime incrementală (Iterative Deepening Search)
 - Căutare uniformă după cost (Uniform-Cost Search)
 - Căutare bidirecțională (Bidirectional Search)

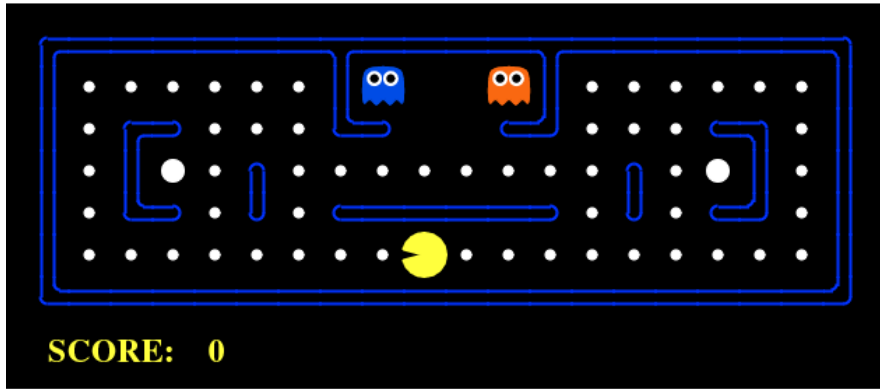
Cuprinsul cursului de azi

1. Agenți inteligenți și mediile în care aceștia funcționează
2. Rezolvarea problemelor prin căutare
 - Formularea unei probleme
 - Graful stărilor
 - Arborele de căutare
 - Căutare într-un arbore de căutare
3. Căutare neinformată
 - Căutare în lățime (Bread-First Search)
 - Căutare în adâncime (Depth-First Search)
 - Căutare în adâncime limitată (Depth-Limited Search)
 - Căutare în adâncime incrementală (Iterative Deepening Search)
 - Căutare uniformă după cost (Uniform-Cost Search)
 - Căutare bidirecțională (Bidirectional Search)

Agenți inteligenți

- agent = entitate care percepe și acționează într-un anumit mediu
 - exemple: om, robot, mașină autonomă, program software
- agenții percep mediul în care se află (starea curentă a mediului) pe baza unor senzori:
 - robot: camera, laser, sonar
 - om: cele 5 simțuri (văz, auz, olfactiv, tactil, papil gustativ)
- agenții acționează asupra mediului folosind actuatori:
 - robot: roți, șine, brațe
 - om: membre (brațe, picioare)
- agenții realizează acțiuni pentru a atinge un scop
- agent inteligent: agenți înzestrați cu o “intelență” care îi ajută să ia deciziile optime/realizeze acțiuni în mediile lor, asemenea oamenilor

Agenți inteligenți și mediile în care aceștia funcționează



1	2	3
8		4
7	6	5

Proprietăți ale mediilor în care funcționează agenții

- **Complet observabil vs parțial observabil:** la orice moment în timp mediul este complet observabil dacă senzorii agentului îi dau acces complet la starea mediului și îi oferă toate aspectele relevante în luarea deciziei optime.
 - jocul de dame vs poker
- **Determinist vs stochastic:** dacă următoarea stare a mediului este în întregime determinată de starea curentă și de acțiunile agentului, mediul este determinist (nu există incertitudine).
 - șah vs pac-man
- **Discret vs continuu:** dacă spațiul stărilor din mediu și numărul de acțiuni ale agentului sunt finite atunci mediul este discret.
 - jocul 8-puzzle vs condusul unei mașini

Proprietăți ale mediilor în care funcționează agenții

- **Episodic vs secvențial**: într-un mediu episodic, experiența agentului este împărțită în episoade independente. Un episod constă în percepție urmată de o singură acțiune. Episoadele următoare nu depind de acțiunile episoadelor anterioare.
 - OCR pe cifre independente (ex. LeNet pe MNIST) vs șah
- **Static vs dinamic**: dacă mediul se schimbă cât timp agentul decide ce acțiuni să facă mediul este dinamic.
 - jocul de șah vs pac-man
- **Agent vs multiagent**: dacă sunt mai mulți agenți mediul este multiagent (o entitate B este considerată agent dacă are un comportament care maximizează o măsură de performanță care depinde de comportamentul (deciziile) agentului A; distingem sisteme multiagent competitive vs sisteme multiagent cooperative).
 - 8-puzzle vs șah

Jocul de dame



Complet observabil vs parțial observabil

Determinist vs stochastic

Discret vs continuu

Episodic vs secvențial

Static vs dinamic

Agent vs multiagent

Complet observabil

Determinist

Discret

Secvențial

Static

Multiagent competitiv

Jocul de poker



Complet observabil vs parțial observabil

Determinist vs stochastic

Discret vs continuu

Episodic vs secvențial

Static vs dinamic

Agent vs multiagent

Parțial observabil

Stochastic

Discret

Secvențial

Static

Multiagent competitiv

8 - puzzle

1	2	3
8		4
7	6	5

Complet observabil vs parțial observabil

Determinist vs stochastic

Discret vs continuu

Episodic vs secvențial

Static vs dinamic

Agent vs multiagent

Complet observabil

Determinist

Discret

Secvențial

Static

Agent

Mașină autonomă



Complet observabil vs parțial observabil

Determinist vs stochastic

Discret vs continuu

Episodic vs secvențial

Static vs dinamic

Agent vs multiagent

Parțial observabil

Stochastic

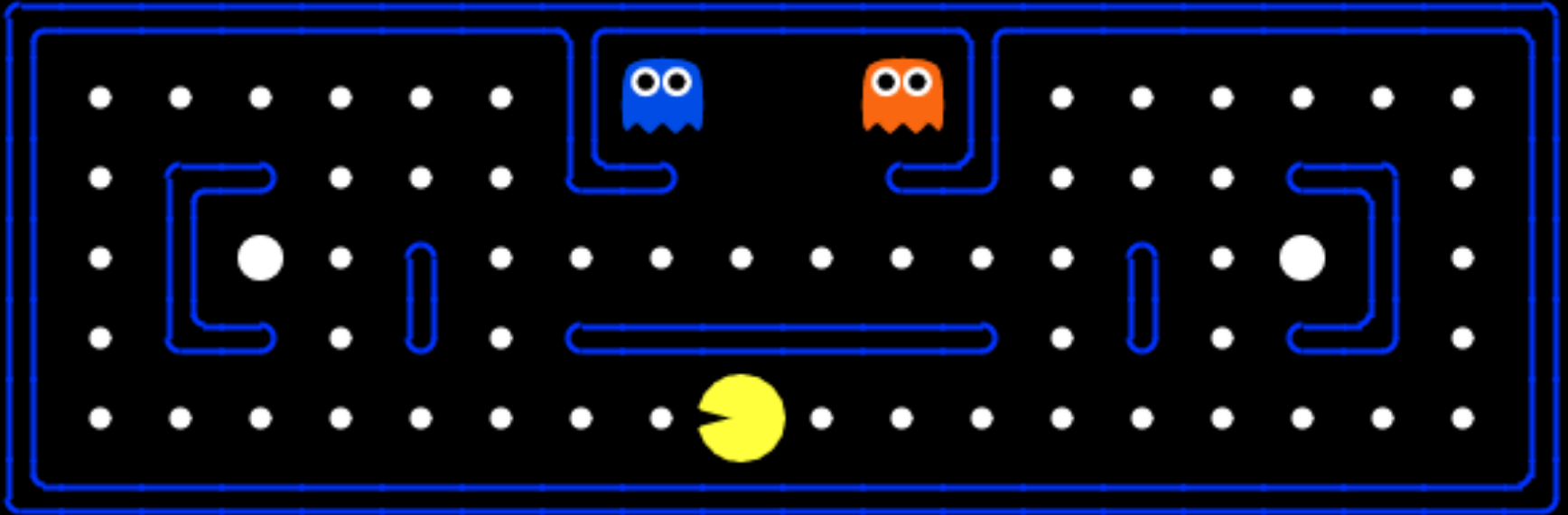
Continuu

Secvențial

Dinamic

Multiagent cooperativ

PAC MAN



SCORE: 0

Complet observabil vs parțial observabil
Determinist vs stochastic
Discret vs continuu
Episodic vs secvențial
Static vs dinamic
Agent vs multiagent

Complet observabil
Stochastic
Discret
Secvențial
Dinamic
Multiagent competitiv

Exemple de medii cu diverse proprietăți

Mediu	Episodic/ Secvențial	Agent / Multiagent	Determinist / Stochastic	Observabil	Static / Dinamic	Discret / Continuu
Rebus	Secvențial	Agent	Determinist	Complet	Static	Discret
Șah	Secvențial	Multiagent	Determinist	Complet	Static	Discret
Table	Secvențial	Multiagent	Stochastic	Complet	Static	Discret
Poker	Secvențial	Multiagent	Stochastic	Parțial	Static	Discret
Starcraft	Secvențial	Multiagent	Stochastic	Parțial	Dinamic	Discret
Lumea reală	Secvențial	Multiagent	Stochastic	Parțial	Dinamic	Continuu

Cuprinsul cursului de azi

1. Agenți inteligenți și mediile în care aceștia funcționează
2. Rezolvarea problemelor prin căutare
 - Formularea unei probleme
 - Graful stărilor
 - Arborele de căutare
 - Căutare într-un arbore de căutare
3. Căutare neinformată
 - Căutare în lățime (Bread-First Search)
 - Căutare în adâncime (Depth-First Search)
 - Căutare în adâncime limitată (Depth-Limited Search)
 - Căutare în adâncime incrementală (Iterative Deepening Search)
 - Căutare uniformă după cost (Uniform-Cost Search)
 - Căutare bidirecțională (Bidirectional Search)

De ce rezolvăm probleme prin căutare?

- Un agent inteligent trebuie să aleagă acțiunile potrivite pentru a-și atinge scopul. Dar cum decide ce să facă?
- Căutarea este procesul prin care agentul explorează, „în minte”, diferite secvențe posibile de acțiuni și consecințele acestora. Pornind din starea inițială, agentul:
 - generează posibile evoluții ale situației,
 - evaluează rezultatele acestora,
 - și selectează o secvență de acțiuni care conduce la o stare scop.
 - dacă există mai multe soluții, agentul poate alege una optimă (de exemplu, cu cost minim).

Rezolvarea problemelor prin căutare

- primul pas: formularea scopului
 - un scop este o mulțime de stări ale universului în care vrem să ajungem
 - poate fi exprimat explicit sau prin intermediul unei funcții de evaluare
- al doilea pas: formularea problemei
 - stare = configurație a lumii înconjurătoare (a universului problemei)
 - ce stări trebuie luate în considerare (abstractizare – elimină detaliile nesemnificative)
 - ce acțiuni (operatori) poate face agentul dintr-o anumită stare
 - acțiunile generează tranziția între stări ale universului
- algoritm de căutare
 - primește ca date de intrare o problemă
 - întoarce ca date de ieșire o soluție sub forma unei secvențe de acțiuni

Formularea unei probleme

- Spațiul stărilor
 - stare = configurație a lumii înconjurătoare
 - mulțimea stărilor în care se poate ajunge plecând din starea inițială, prin intermediul oricărei secvențe de acțiuni
 - ce stări trebuie luate în considerare (abstractizare)
 - stare inițială
 - stare/stări scop
- Acțiuni
 - ce acțiuni (operatori) poate face agentul dintr-o anumită stare
 - acțiunile generează tranziția între stări ale universului
 - uneori acțiunile au un cost asociat
- Funcția succesor
 - $SUCCESSOR(x, a)$ = mulțimea stărilor în care se poate ajunge din starea x prin acțiunea a

Rezolvarea unei probleme

- Soluție = drum de la starea inițială la o stare scop
 - drum = secvență ordonată de acțiuni
- Costul unei soluții = costul drumului
 - suma costurilor acțiunilor din secvența ordonată
 - acțiunile au de obicei cost > 0
- Soluția optimă are costul cel mai mic dintre soluțiile posibile

Problema 8-puzzle

Descrierea problemei

Pe o tablă 3×3 se găsesc 8 piese, numerotate de la 1 la 8. La fiecare moment o singură piesă se poate mișca cu o poziție, pe orizontală sau verticală, în limitele cadrului tablei, în locul rămas liber (poziția roșie). Se dă o configurație inițială și una finală a tablei. Trebuie să se găsească secvența de mutări care să aducă piesele din configurația inițială în cea finală.

2		3
1	8	4
7	6	5

Stare inițială

[2, X, 3, 1, 8, 4, 7, 6, 5]



1	2	3
8		4
7	6	5

Stare scop

[1, 2, 3, 8, X, 4, 7, 6, 5]

Formularea problemei 8-puzzle

- Spațiul stărilor

- stare = configurație a pătratului 3×3
- $9!/2$ stări posibile
- stare inițială
- stare scop

2		3
1	8	4
7	6	5

Stare inițială

[2, X, 3, 1, 8, 4, 7, 6, 5]

1	2	3
8		4
7	6	5

Stare scop

[1, 2, 3, 8, X, 4, 7, 6, 5]

- Acțiuni

- STÂNGA, JOS, DREAPTA, SUS
- fiecare acțiune (mutare) are cost = 1

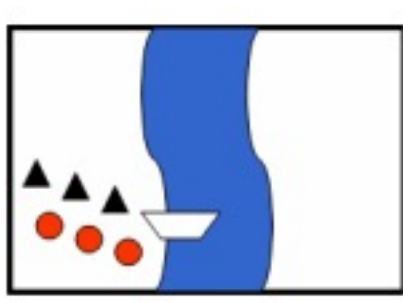
- Funcția succesori

- $SUCCESSOR([X, a, b, c, d, e, f, g, h], STANGA) = \text{NU EXISTA}$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], SUS) = \text{NU EXISTA}$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], DREAPTA) = [a, X, b, c, d, e, f, g, h]$
- $SUCCESSOR([X, a, b, c, d, e, f, g, h], JOS) = [c, a, b, X, d, e, f, g, h]$
- ...

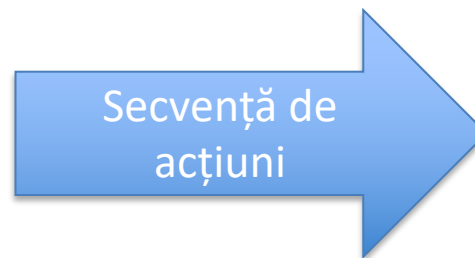
Problema misionarilor și a canibalilor

Descrierea problemei

Trei misionari și **trei canibali** se află la marginea unui râu, cu scopul de a trece pe celălalt mal. Ei au la dispoziție o barcă de două persoane. Dacă la un moment dat, pe un mal sau pe celălalt, numărul canibalilor este mai mare decât cel al misionarilor, canibalii îi vor mânca pe misionari. Problema constă în a afla cum pot trece cele 6 persoane în deplină siguranță de pe un mal pe celălalt😊. Barca nu merge singură, este nevoie de cel puțin o persoană în barcă.



Stare inițială

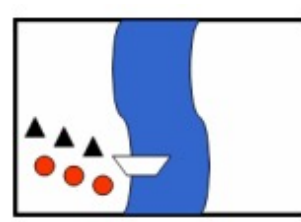


Stare scop

Formularea problemei misionarilor și a canibalilor

- Spațiul stărilor

- triplet (m, c, b) : m = numărul de misionari pe malul stâng, c = numărul de canibali pe malul stâng, b = prezența bărcii (1 sau 0) pe malul stâng
- stare inițială: $(3,3,1)$
- stare scop: $(0,0,0)$



Stare inițială



Stare scop

- Acțiuni

- pot duce de pe un mal către celălalt mal 1 sau 2 misionari, 1 sau 2 canibali, 1 misionar + 1 canibal ($1m, 2m, 1c, 2c, 1m+1c$)

- Funcția succesori

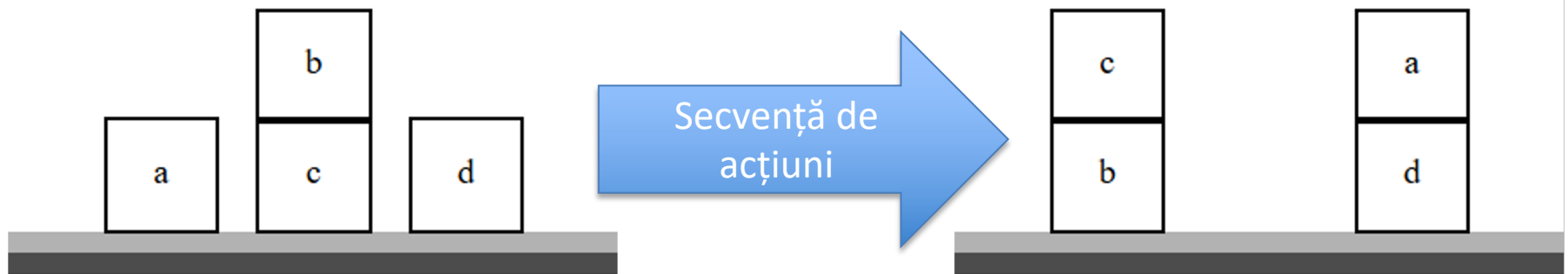
- $\text{Successor}((m, c, b), 1c) = (m, c + 1 - 2b, 1 - b), \dots$
- $\text{Successor}((m, c, b), 1m) = (m + 1 - 2b, c, 1 - b), \dots$

Laborator – problema blocurilor

Descrierea problemei

Considerăm un ansamblu ordonat de stive $LS = \{S_1, S_2, S_3, \dots, S_n\}$, fiecare stivă conținând un număr finit de blocuri din mulțimea $MB = \{B_1, B_2, B_3, \dots, B_m\}$. Unele stive pot fi inițial vide. Fiecare bloc are o poziție bine definită: el se află fie pe o altă stivă (deasupra altui bloc), fie direct la baza stivei. La un moment dat, **doar un singur bloc** poate fi mutat, și anume blocul aflat **în vârful unei stive**. O **mutare** validă constă în ridicarea unui bloc aflat în vârful unei stive și plasarea lui fie pe vârful altei stive fie pe o stivă vidă.

Scopul problemei este de a determina o **secvență de mutări** care transformă o **configurație inițială** dată (dispunerea blocurilor în stive) într-o **configurație finală** dorită, în care blocurile sunt aranjate conform unei ordini sau structuri țintă.

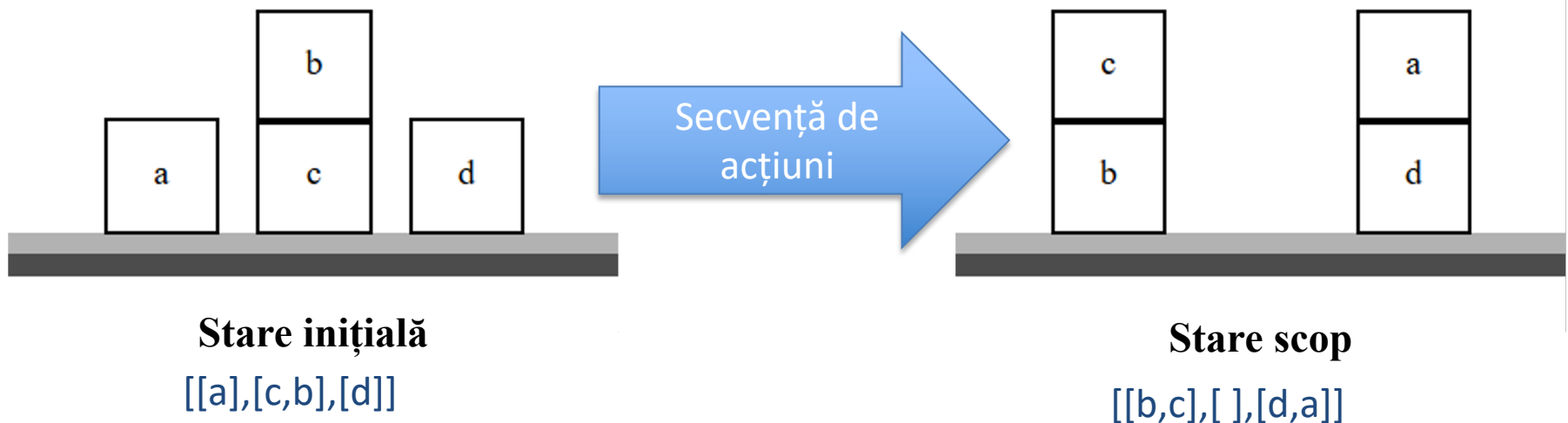


Laborator – problema blocurilor

Descrierea problemei

Considerăm un ansamblu ordonat de stive $LS = \{S_1, S_2, S_3, \dots, S_n\}$, fiecare stivă conținând un număr finit de blocuri din mulțimea $MB = \{B_1, B_2, B_3, \dots, B_m\}$. Unele stive pot fi inițial vide. Fiecare bloc are o poziție bine definită: el se află fie pe o altă stivă (deasupra altui bloc), fie direct la baza stivei. La un moment dat, **doar un singur bloc** poate fi mutat, și anume blocul aflat **în vârful unei stive**. O **mutare** validă constă în ridicarea unui bloc aflat în vârful unei stive și plasarea lui fie pe vârful altei stive fie pe o stivă vidă.

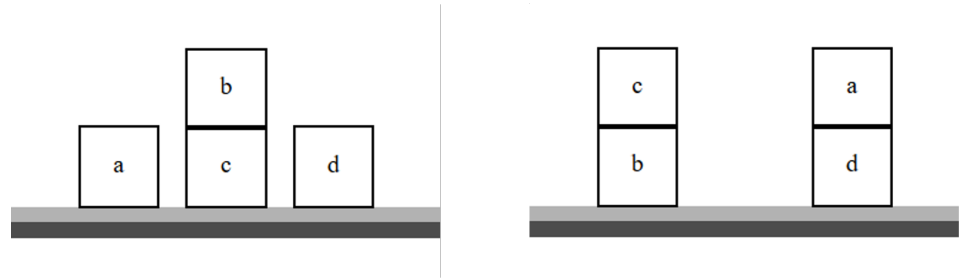
Scopul problemei este de a determina o **secvență de mutări** care transformă o **configurație inițială** dată (dispunerea blocurilor în stive) într-o **configurație finală** dorită, în care blocurile sunt aranjate conform unei ordini sau structuri țintă.



Formularea problemei blocurilor

- Spațiul stărilor

- stare = listă ordonată de stive $s = [S_1, S_2, \dots, S_n]$, unde fiecare stivă S_i este o listă (eventual vidă) de blocuri $S_i = [B_{i1}, B_{i2}, \dots, B_{ik}]$, cu ultimul bloc (B_{ik}) fiind vârful stivei
- stare inițială $[[a],[c,b],[d]]$
- stare scop $[[b,c],[],[d,a]]$



- Acțiuni

- o acțiune este de forma $a = \text{move}(B, i, j)$, unde B este un bloc aflat în vârful stivei S_i , iar $j \neq i$ este indicele stivei S_j pe care blocul va fi mutat

- Funcția succesor

- $\text{Succesor}([\dots, S_i, \dots, S_j, \dots], \text{move}(B, i, j)) = [\dots, S_i - [B], \dots, S_j + [B], \dots]$

Probleme de căutare cu spațiu de stări infinit

Scrierea numerelor naturale folosind numai 3 operații aritmetice

Descrierea problemei

Conjectura lui Donald Knuth (1964): pornind de la numărul 4 și folosind numai 3 operații aritmetice (factorial, radical, parte întreagă) putem obține orice numărul natural nenul.

$$\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \rfloor = 5.$$

- Problemă de căutare
 - stări (configurații ale lumii) = numere naturale
 - acțiuni: aplicarea operațiilor de factorial (pe numere întregi), radical, parte întreagă
 - stare inițială: 4
 - funcția succesori: definită de cele operații matematice
 - stare scop: numărul natural pe care vrem să îl obținem

Probleme de căutare cu spațiu de stări infinit

Scrierea numerelor naturale folosind numai 3 operații aritmetice

Descrierea problemei

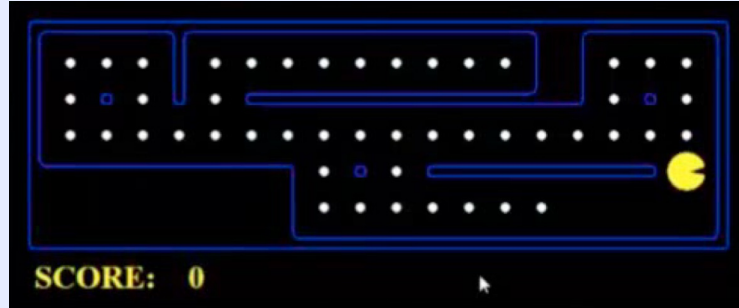
Conjectura lui Donald Knuth (1964): pornind de la numărul 4 și folosind numai 3 operații aritmetice (factorial, radical, parte întreagă) putem obține orice numărul natural nenul.

$$\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \rfloor = 5.$$

- În obținerea lui 5 avem $(4!)! = 620,448,401,733,239,439,360,000$
- Nu există o limită superioară a numerelor pe care le putem obținem ca stări intermediare pentru a ajunge la o stare scop.
- Spațiu de stări infinit
 - probleme pentru anumite tipuri de căutări (căutarea în adâncime)

Ce conține spațiul stărilor?

O **stare a lumii** include fiecare detaliu al mediului (universul problemei)



O **stare pentru căutare** păstrează numai informația relevantă astfel încât evoluția mediului să poată fi determinată doar din starea curentă și acțiunea curentă (abstractizare)

- Problema: vreau să ajung într-un anumit punct (traseu)

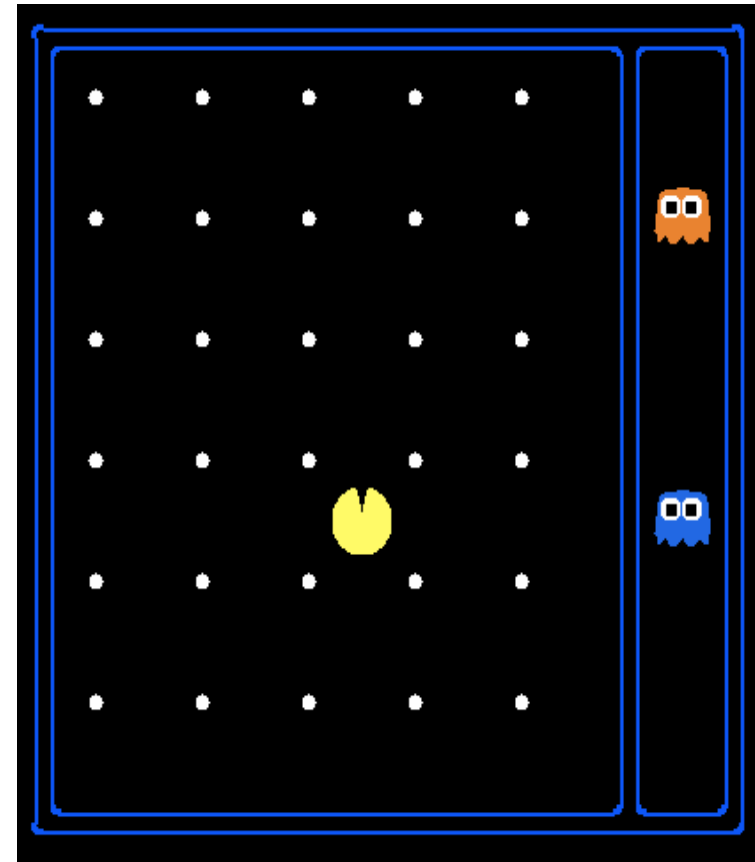
- stări: poziția (x,y)
- acțiuni: Nord Sud Est Vest
- funcție succesori: actualizează numai poziția
- test scop: (x,y)=END ?

- **Problemă: colectarea tuturor punctelor din mediu**

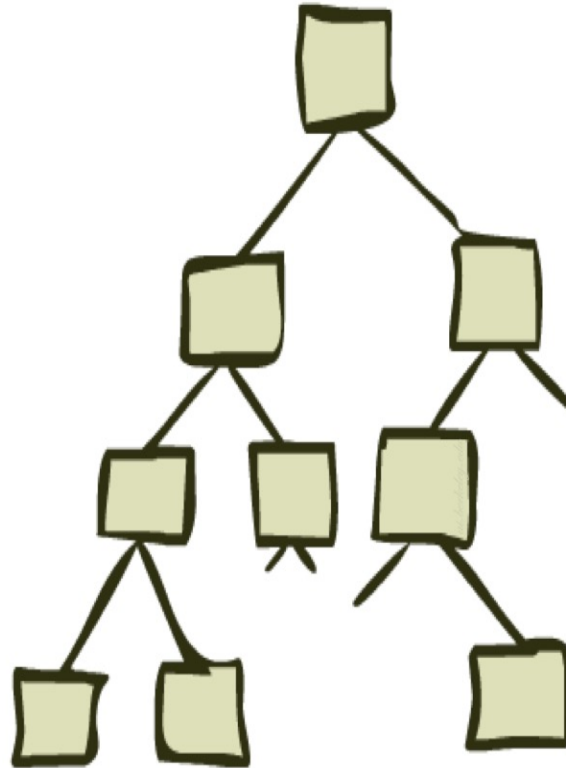
- stări: {(x,y), var. booleană pentru puncte albe}
- acțiuni: Nord Sud Est Vest
- succesori: actualizează poziția + posibil variabila booleană
- test scop: toate variabilele booleene sunt false

Dimensiunea spațiului stărilor

- Stările lumii:
 - pozițiile agentului: 120
 - poziția punctelor albe: 30
 - poziții fantome: 12
- Câte:
 - stări ale lumii?
 $120 \times (2^{30}) \times (12^2)$ (margine superioară)
 - stări pentru problema de a ajunge într-un anumit punct?
120
 - stări pentru problema de colectare a tuturor punctelor din mediu?
 $120 \times (2^{30})$



Graful de stări și arborele de căutare

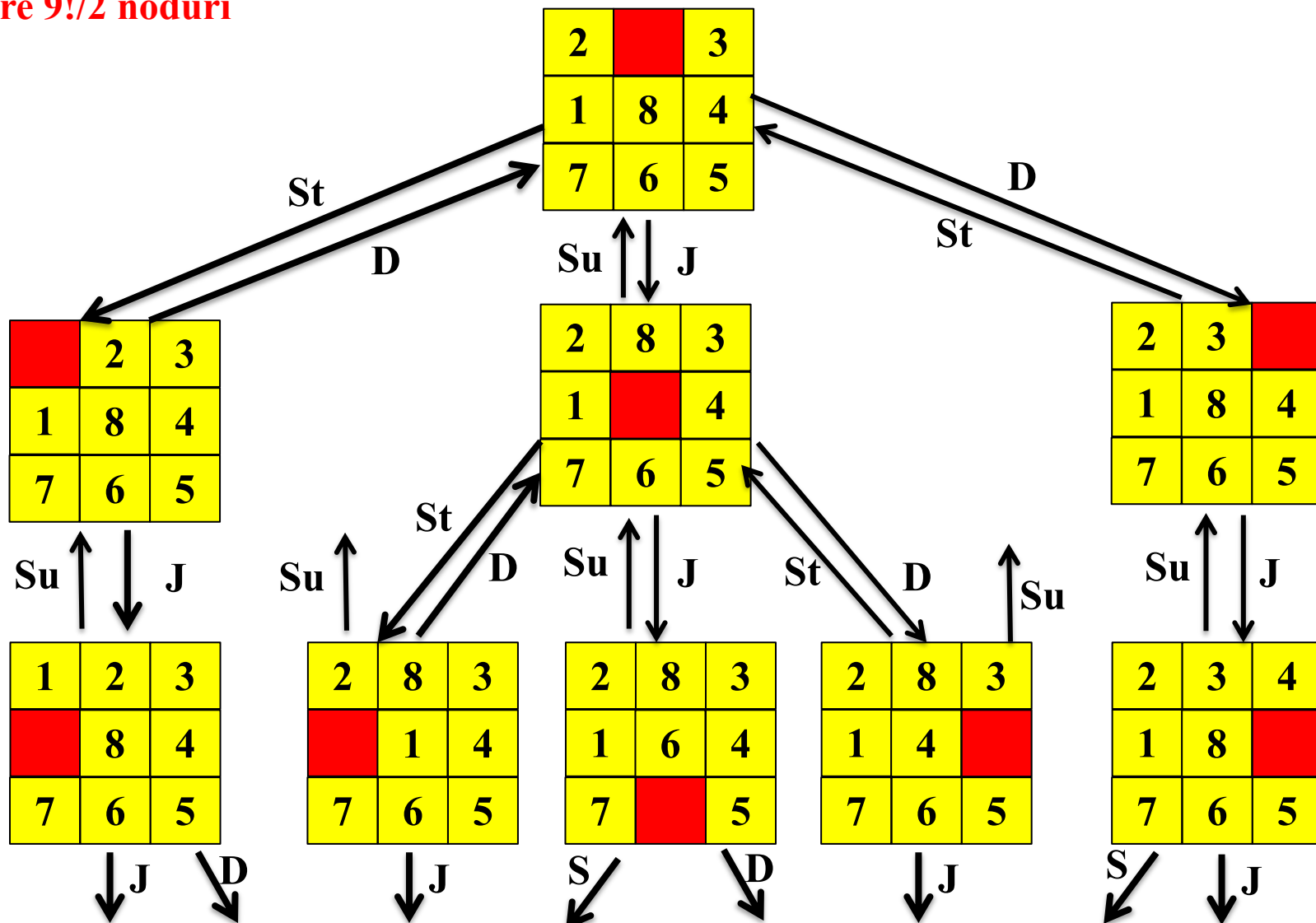


Graful de stări al unei probleme

- Graful de stări este un graf orientat care conține toate stările accesibile din starea inițială, iar arcele reprezintă tranzițiile generate de acțiuni.
- Fiecare nod din graf reprezintă o stare (**un nod=o stare**).
- Arcele modelează funcția succesor:
 - arată în ce stări putem ajunge din starea curentă pentru o anumită acțiune
 - dacă acțiunile sunt simetrice (reversibile), pot exista arce în ambele sensuri între două stări.

Graful de stări al problemei 8 - puzzle

Are $9!/2$ noduri



Arborele de căutare al unei probleme

- Graful de stări $\rightarrow$ descrie problema
- Arborele de căutare $\rightarrow$ descrie explorarea problemei

Arborele de căutare este un arbore orientat generat prin explorarea grafului de stări organizat pe nivele:

- nivelul 0 = rădăcina (starea inițială)
- nivelul i = nod obținut prin aplicarea a i acțiuni (i tranziții) pornind din starea inițială

Fiecare nod din arborele de căutare reprezintă o soluție parțială (un drum de la rădăcină până la acel nod). Fiecare arc reprezintă aplicarea unei acțiuni (un pas în construirea soluției)

În **graful de stări**: o stare apare o singură dată.

În **arborele de căutare**: aceeași stare poate apărea de mai multe ori (prin drumuri diferite).

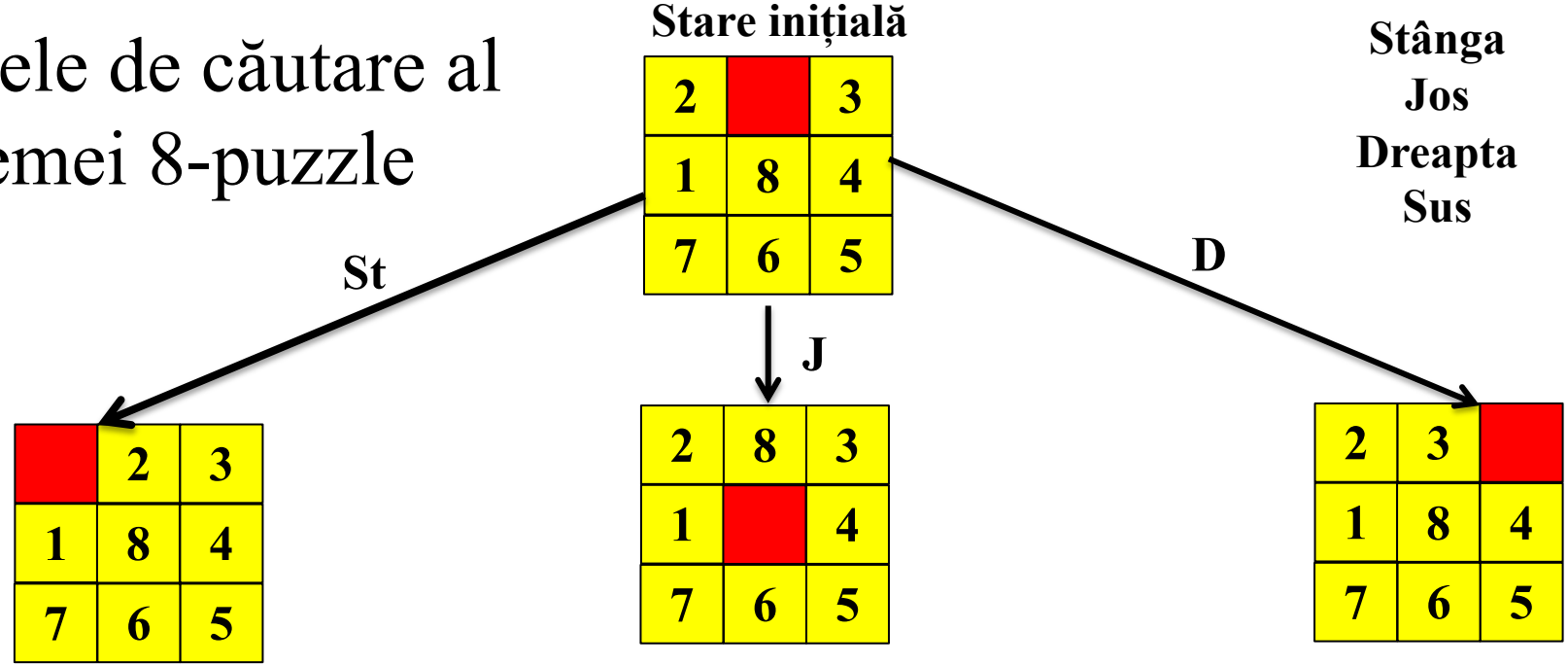
Arborele de căutare al problemei 8-puzzle

Stare inițială

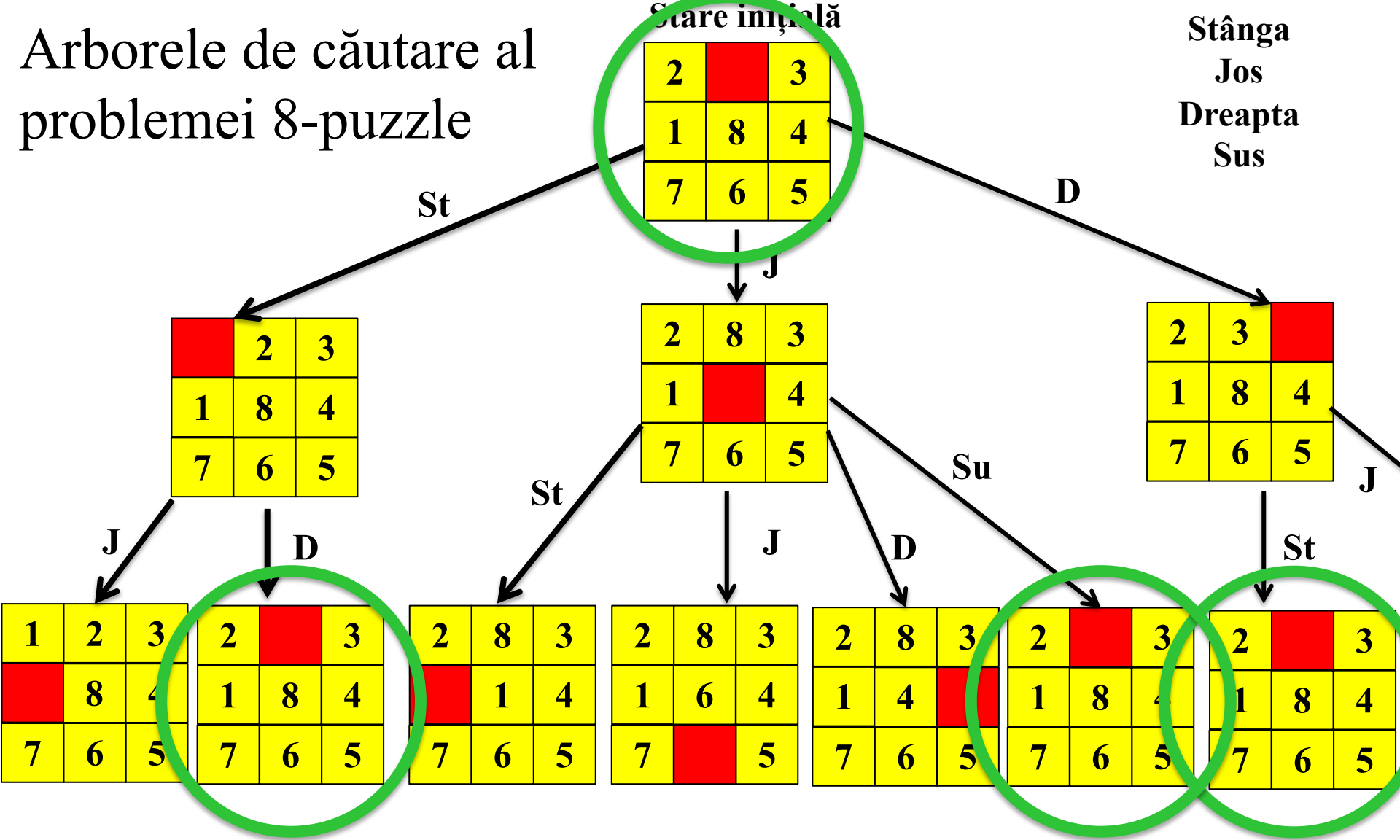
2		3
1	8	4
7	6	5

Stânga
Jos
Dreapta
Sus

Arborele de căutare al problemei 8-puzzle

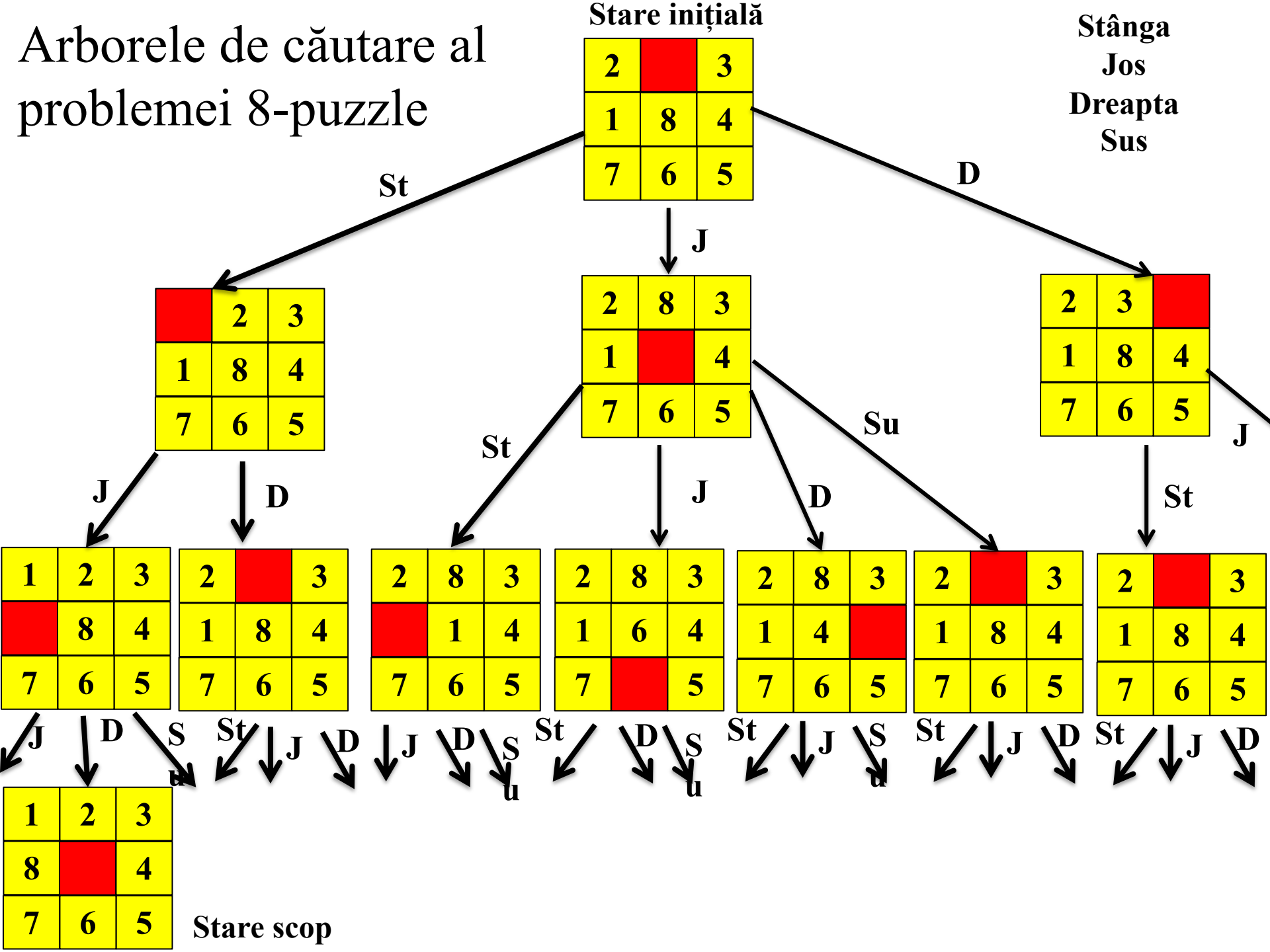


Arborele de căutare al problemei 8-puzzle



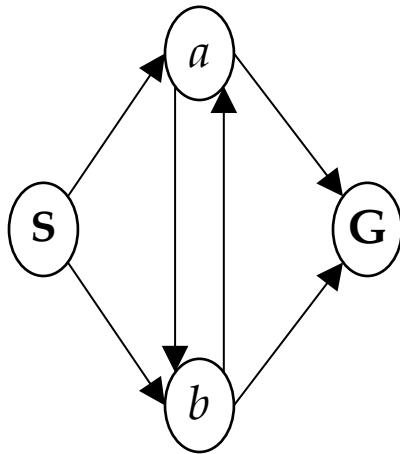
În arborele de căutare, noduri diferite pot corespunde aceleiași stări din graful de stări, deoarece sunt generate prin secvențe diferite de acțiuni.

Arborele de căutare al problemei 8-puzzle



Graful de stări vs. Arborele de căutare

Considerăm graful cu 4 stări de mai jos:



s – stare inițială

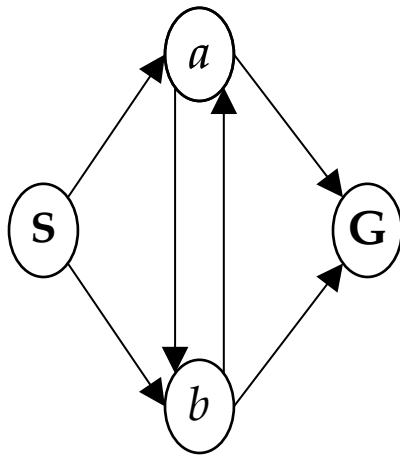
G – stare scop (goal)

Cât de mare este arborele de căutare (pornim din s) ?



Graful de stări vs. Arborele de căutare

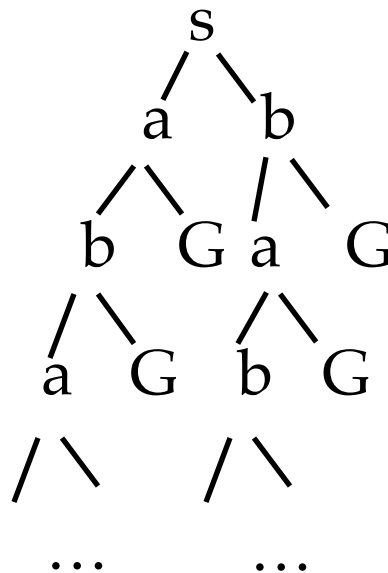
Considerăm graful cu 4 stări de mai jos:



s – stare inițială

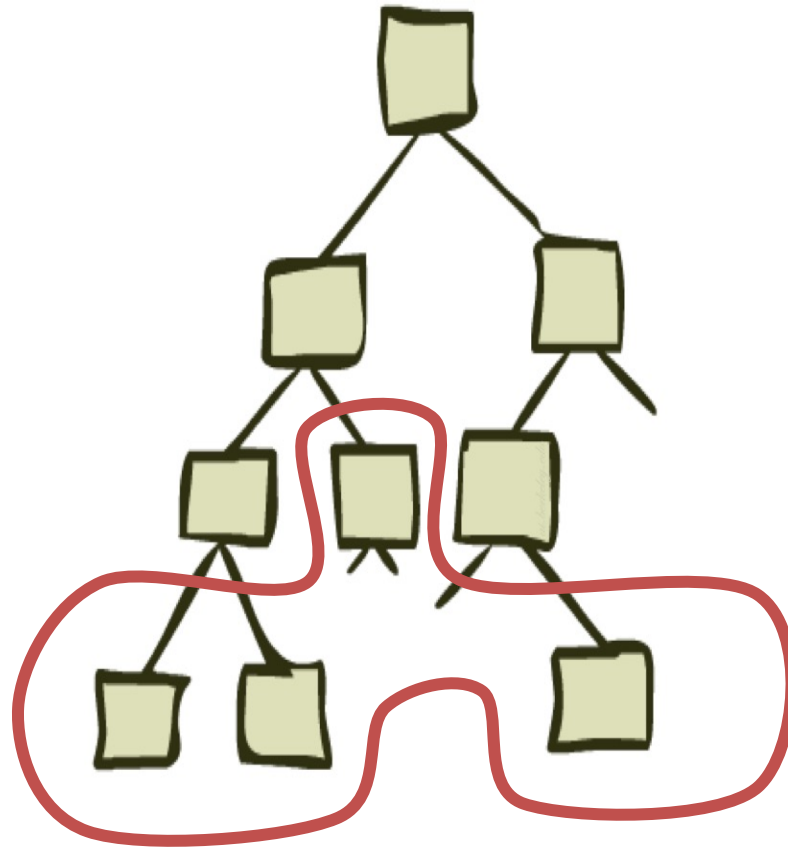
G – stare scop (goal)

Cât de mare este arborele de căutare (pornim din s) ?

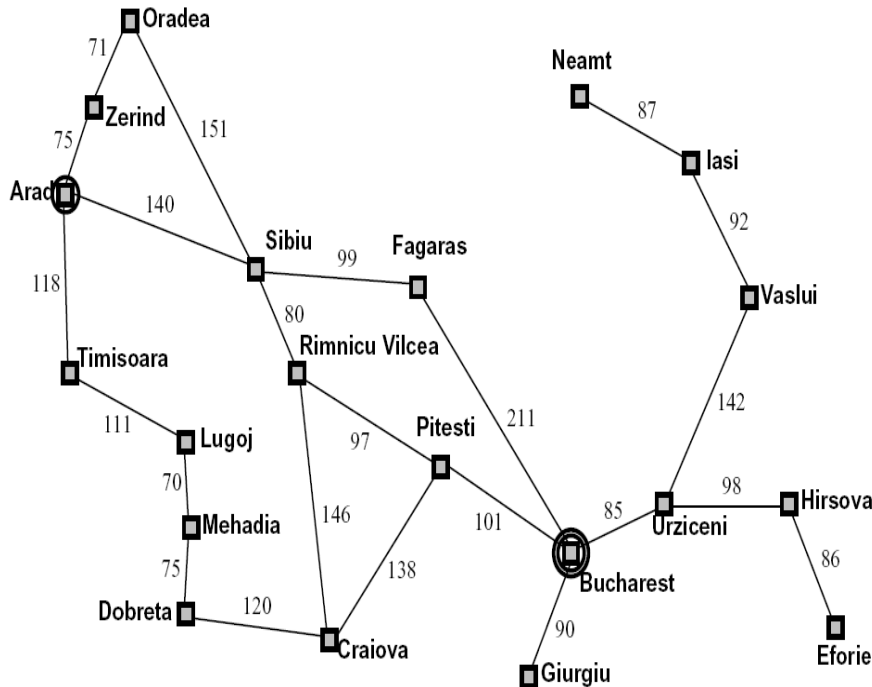


Important: foarte multe structuri repetitive în arborele de căutare

Căutare într-un arbore de căutare



Exemplu de căutare: Arad-București



- Spațiul stărilor:
 - orașe
- Acțiuni
 - mergi de la un oraș la un alt oraș pe un drum;
 - costul drumului = distanța
- Funcție succesor:
 - pornind dintr-un oraș pe un drum ajung la alt oraș
- Stare inițială:
 - Arad
- Scop care poate fi verificat:
 - stare == Bucharest?
- Soluție?

Căutare într-un arbore de căutare

- Căutarea constă în generarea și selectarea nodurilor din arborele de căutare până când este găsită o stare scop.
- Fiecare nod reprezintă o soluție parțială (un drum de la starea inițială până la starea curentă).
- Un nod este o structură de date care conține următoarele informații:

nod = (stare, părinte, acțiune, cost)

- **stare** – configurația curentă a problemei;
- **părinte** – o referință (pointer) către nodul din care a fost generat;
- **acțiune** – acțiunea aplicată pentru a ajunge în această stare;
- **cost** (notat de obicei cu $g(n)$) – costul drumului de la starea inițială până la acest nod n .

Structura nodului permite reconstruirea soluției prin urmărirea referințelor către părinți până la rădăcină, precum și calcularea costului total al drumului ($g(n)$).

Rezolvarea unei probleme prin căutare

După formularea problemei (stări, acțiuni, cost, stare inițială, scop, funcția succesor), trebuie să găsim o **soluție**.

Soluție = **secvență de acțiuni** care conduce de la starea inițială la o stare scop. Toate secvențele posibile de acțiuni pornind din starea inițială formează un arbore de căutare în care:

- rădăcina = starea inițială
- nodurile = (stare, părinte, acțiune, cost)
- arcele = aplicarea unei acțiuni

Arborele de căutare este generat progresiv, nu este construit integral de la început.

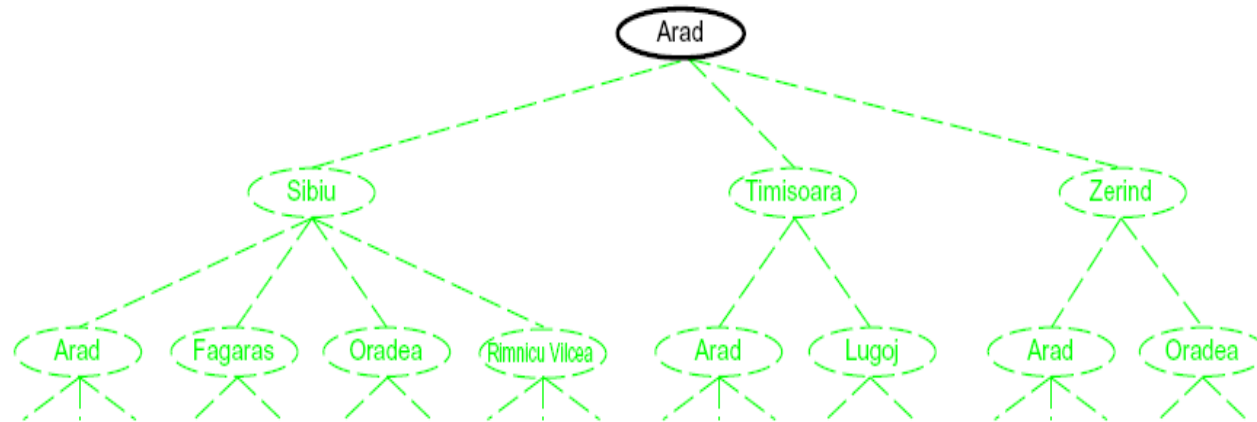
Rezolvarea unei probleme prin căutare

Algoritmul de căutare operează asupra unei structuri numite **frontieră** (denumită și **fringe** sau **open list**), care conține inițial un singur nod, corespunzător stării inițiale (rădăcina arborelui).

Algoritmul repetă următorii pași:

1. selectează un nod din frontieră (conform unei strategii).
2. testează dacă starea asociată nodului este stare scop.
3. dacă nu e stare scop, expandează nodul și adaugă succesorii în frontieră.

Exemplu de căutare într-un arbore de căutare

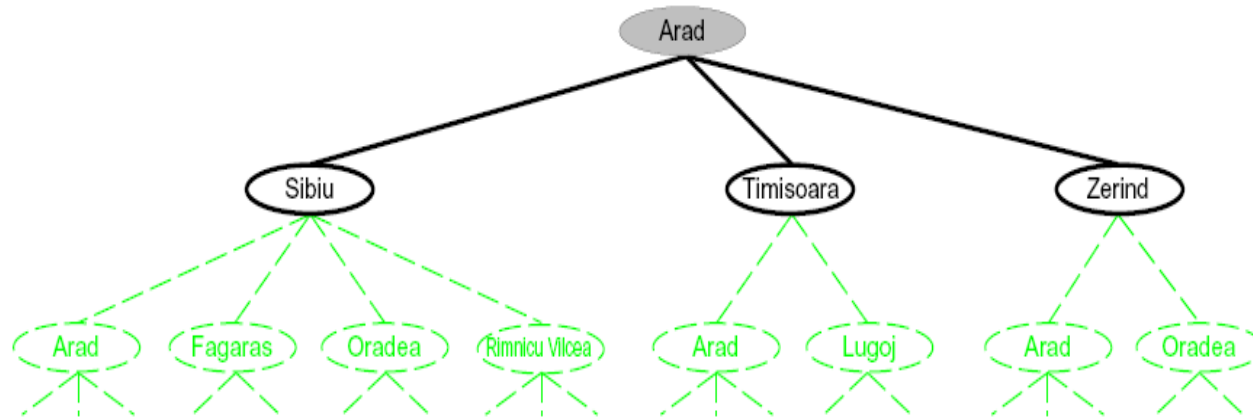


Frontiera (Open List): {Arad}

Algoritmul repetă următorii pași:

1. selectează un nod din frontieră (conform unei strategii).
2. testează dacă starea asociată nodului este stare scop.
3. dacă nu e stare scop, expandează nodul și adaugă succesorii în frontieră.

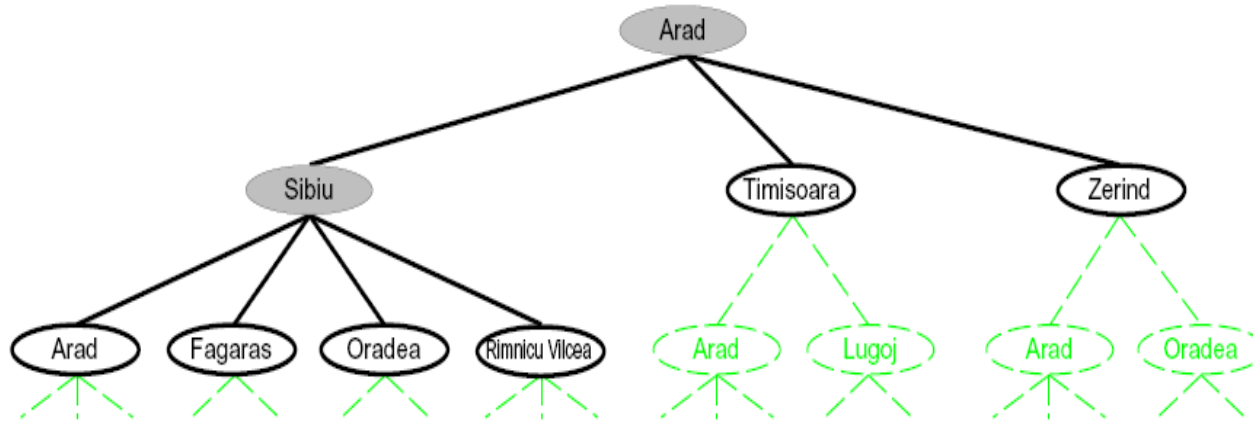
Exemplu de căutare într-un arbore de căutare



Frontiera (Open List): {Sibiu, Timișoara, Zerind}

Nodurile expandate sunt colorate în gri; nodurile din frontieră sunt evidențiate prin contur îngroșat; nodurile încă negenerate sunt reprezentate cu linii întrerupte.

Exemplu de căutare într-un arbore de căutare



Frontiera (Open List): {Timișoara, Zerind, Arad, Făgăraș, Oradea, Râmnicu Vâlcea}

Nodurile expandate sunt colorate în gri; nodurile din frontieră sunt evidențiate prin contur îngroșat; nodurile încă negenerate sunt reprezentate cu linii întrerupte.

Algoritmul general de căutare în arbore

funcția **TREE-SEARCH** (**problema**, **strategie**) **returnează** o soluție sau eșec

inițializează **frontiera** folosind **starea inițială a problemei**

ciclează

dacă **frontiera** este vidă **atunci returnează** eșec

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

dacă **nodul** conține o **stare scop**

atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesori generate în **frontieră**

În algoritmul de mai sus, fiecare nod din arbore este unic și corespunde unui drum unic de la rădăcină (starea inițială) până la o anumită stare.

- fiecare nod reprezintă o secvență unică de acțiuni.
- două noduri diferite pot conține aceeași stare, dar sunt considerate noduri diferite deoarece au fost obținute prin drumuri diferite.

Algoritmul general de căutare în arbore

funcția **TREE-SEARCH** (**problema**, **strategie**) **returnează** o soluție sau eșec

inițializează **frontiera** folosind **starea inițială a problemei**

ciclează

dacă frontiera este vidă **atunci returnează** eșec

alege un **nod frunză** conform **strategiei** și elimină-l din **frontieră**

dacă nodul conține o **stare scop**

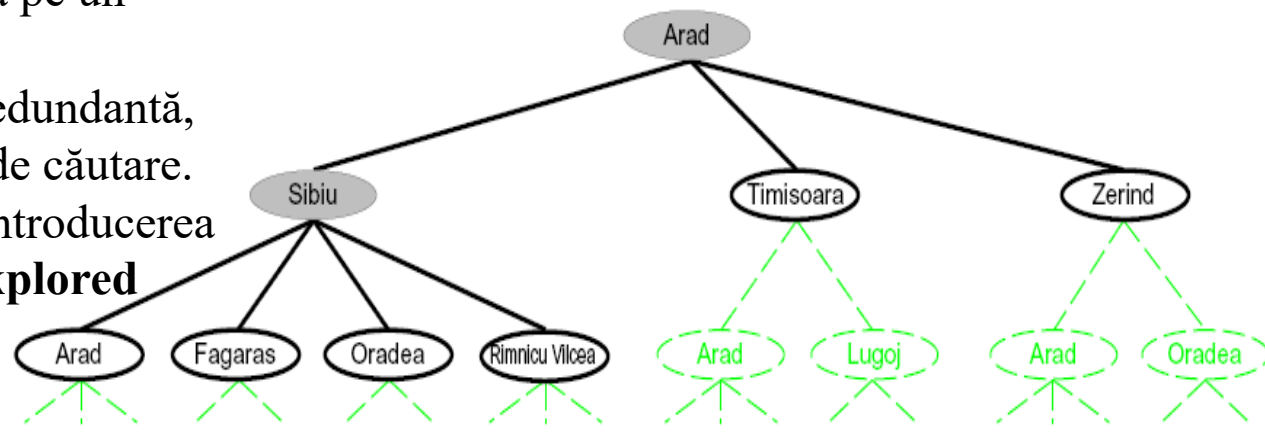
atunci returnează soluția corespunzătoare

expandează nodul ales, adăugând nodurile succesor generate în **frontieră**

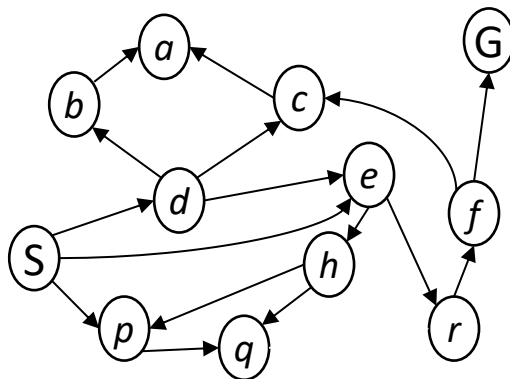
Aceeași stare poate fi reîntâlnită pe un drum, ceea ce duce la:

- cicluri (bucle) și explorare redundantă,
- creșterea inutilă a arborelui de căutare.

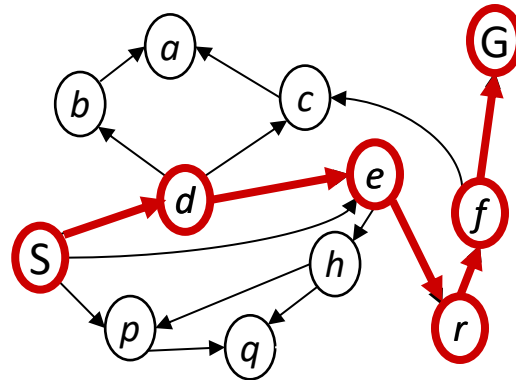
Vom modifica algoritmul prin introducerea **mulțimii stărilor explorate (explored /closed set)**.



Arbori de căutare - exemple

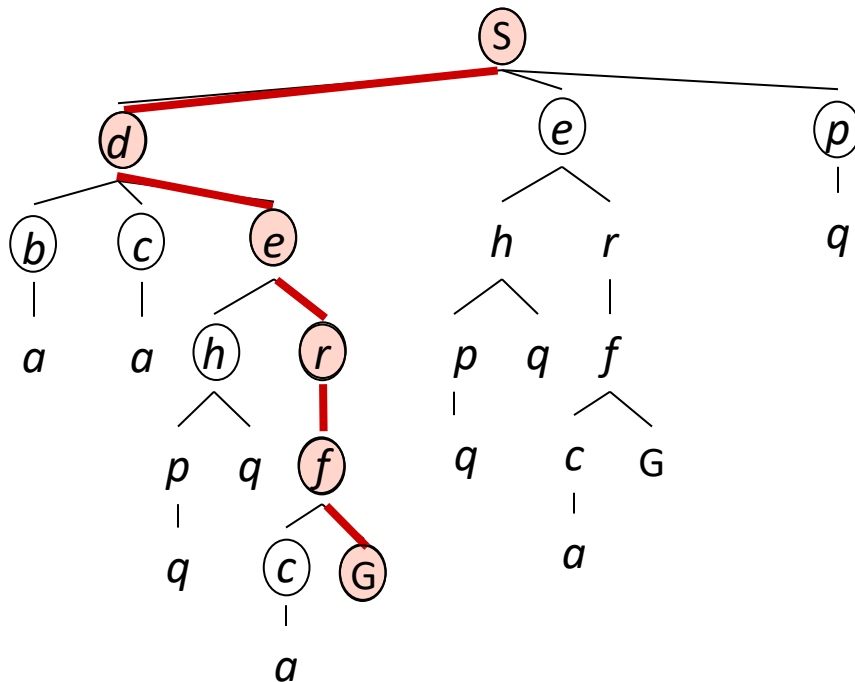


Arbori de căutare - exemple



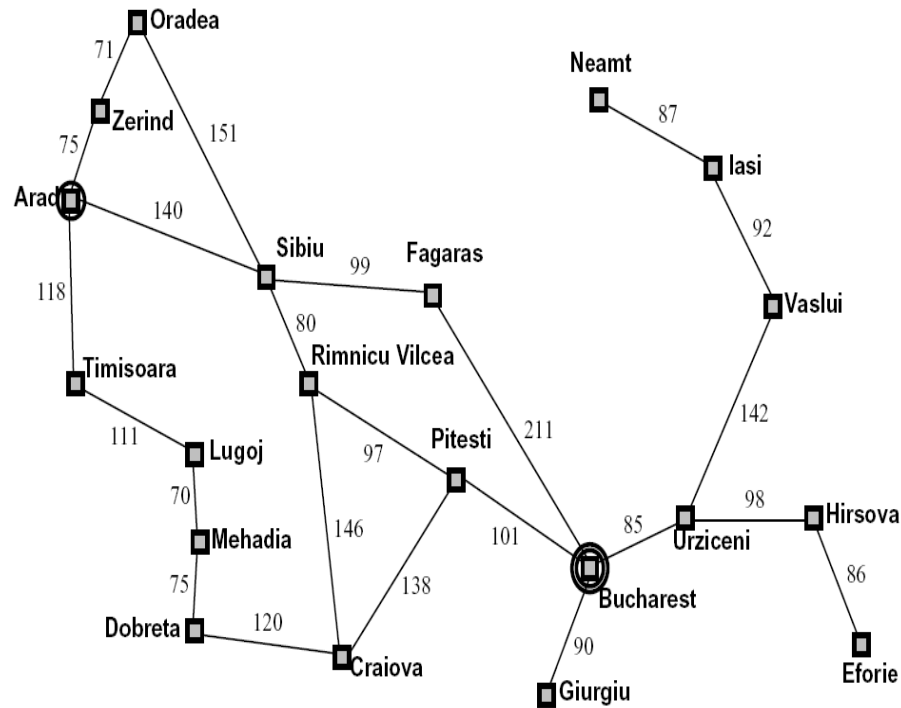
Arborele de căutare

Frontiera (conține noduri = soluții parțiale)



~~s~~
~~s → d~~
s → e
s → p
s → d → b
s → d → c
~~s → d → e~~
s → d → e → h
~~s → d → e → r~~
~~s → d → e → r → f~~
s → d → e → r → f → c
~~s → d → e → r → f → G~~

Arbori de căutare - exemple



(a) Stare inițială



(b) Expandăm nodul Arad



(c) Expandăm nodul Sibiu



Trei tipuri de noduri:

- noduri vizitate
- noduri din frontieră
- noduri negenerate



Căutare

- Problemă de căutare
 - stări (configurații ale lumii)
 - stare inițială și stare scop
 - acțiuni și costuri
 - funcția succesor
- Arbore de căutare
 - noduri: soluții parțiale
 - arce: acțiuni
 - o soluție parțială are asociat un cost (suma costurilor acțiunilor = arcele)
- Strategie de căutare
 - construiește un arbore de căutare
 - alege ordinea în care explorează/expandează nodurile din frontieră
 - optimale dacă găsesc soluțiile de cost minim

